MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Verification Using SystemVerilog
Internal Assessment

DESIGN AND VERIFICATION OF A 16-BIT SYNCHRONOUS UP/DOWN COUNTER

Submitted by

Registration Number	Roll No.	Name
240959024	10	Garv Kishore Gupta

SCHOOL OF ELECTRICAL AND ELECTRONICS ENGINEERING
MANIPAL-576104, KARNATAKA, INDIA
NOVEMBER 2026

ABSTRACT

This report presents the design and class-based verification of a 16-bit synchronous up/down counter using SystemVerilog. The counter updates on the rising edge of a common clock and supports an active-high synchronous reset. The direction input selects increment or decrement, and the 16-bit output wraps naturally at the limits of its unsigned range.

The verification environment follows the same architecture as the JK flip-flop testbench: a transaction class, generator, driver, monitor and scoreboard communicate through typed mailboxes. An interface groups the DUT signals, while virtual interface handles allow the driver and monitor to access the instantiated interface.

The stimulus consists of seven directed transactions followed by twenty-four randomized direction transactions. Directed tests exercise reset, underflow, overflow, consecutive increments, a decrement and reset from a nonzero state. The scoreboard maintains an independent expected count and compares it against the observed output after each rising clock edge.

The supplied EPWave capture shows the expected directed transitions and subsequent changes in counting direction. It serves as visual evidence of the displayed run. A console transcript was not supplied, so no measured PASS/FAIL total or functional coverage percentage is claimed. The report includes the design methodology, waveform analysis and extensively commented SystemVerilog listings.

Tools/resources: SystemVerilog, EDA Playground/EPWave, the supplied counter code and waveform capture.

CONTENT

Abstract

Chapter 1 INTRODUCTION

1.1 Introduction
1.2 Objectives

Chapter 2 METHODOLOGY

2.1 Design Methodology
2.2 Verification Architecture
2.3 Stimulus and Timing

Chapter 3 RESULT ANALYSIS

3.1 Output Waveform
3.2 Directed Sequence Analysis
3.3 Verification Scope

Chapter 4 CONCLUSION

4.1 Work Conclusion

Annexure A SYSTEMVERILOG CODE

A.1 RTL - design.sv
A.2 Interface - testbench.sv
A.3 Transaction class
A.4 Generator class
A.5 Driver class
A.6 Monitor class
A.7 Scoreboard class
A.8 Top-level testbench

CHAPTER 1
INTRODUCTION

1.1 Introduction

A counter is a sequential circuit that advances through a numerical sequence in response to clock pulses. A 16-bit counter represents 65,536 states, from decimal 0 to 65,535 (hexadecimal 0000 to FFFF). An up/down counter can move through this sequence in either direction.

In a synchronous counter, all state bits are updated under the same active clock edge. In this design, up_down = 1 increments the count and up_down = 0 decrements it. There is no count-enable input: whenever reset is inactive, the counter changes on every rising edge. Reset is synchronous and therefore clears the output only at a rising edge while rst is high.

Verification must check both ordinary counting and boundary behavior. Underflow from zero produces FFFF, and overflow from FFFF produces zero. A class-based testbench separates stimulus generation, signal driving, observation and response checking so that each component has a clear responsibility.

1.2 Objectives

• Implement a synthesizable-style 16-bit synchronous up/down counter with reset.

• Retain the transaction, generator, driver, monitor and scoreboard architecture used for the JK flip-flop.

• Check increment, decrement, direction reversal, wraparound and reset priority.

• Apply stimulus on falling clock edges and observe the updated response after rising edges.

• Compare the DUT output with a separately maintained expected count and record signals for waveform analysis.

CHAPTER 2
METHODOLOGY

2.1 Design Methodology

Module counter_16 has three one-bit inputs (clk, rst and up_down) and one 16-bit output (count). An always_ff block models the sequential behavior. Nonblocking assignments schedule the new register value, and the reset branch takes priority over both counting directions.

rst	up_down	Operation at rising edge
1	X (ignored)	count = 0000
0	1	count = (count + 1) modulo 65,536
0	0	count = (count - 1) modulo 65,536

The 16-bit destination retains only the 16-bit result of each operation. No explicit boundary detector is needed: FFFF + 1 becomes 0000 and 0000 - 1 becomes FFFF. Here X in the operation table means the direction is irrelevant when reset is asserted; it is not a stimulus requirement.

2.2 Verification Architecture

Component	Responsibility
Transaction	Carries rst, up_down and the observed count; provides display() and copy().
Generator	Queues directed tests and randomized direction tests in G2D.
Driver	Receives G2D transactions and drives inputs at falling edges.
Interface and DUT	Share clock, reset, direction and count signals.
Monitor	Captures inputs at rising edges, then samples count after 1 ns; sends observations through M2S.
Scoreboard	Maintains expected_count, waits for reset initialization and compares every received sample.

Communication path: generator → G2D → driver → interface/DUT → monitor → M2S → scoreboard. Both mailboxes carry transaction object handles. copy() allocates a new object so later edits to the generator's reusable transaction do not change already queued stimulus.

CHAPTER 2
METHODOLOGY (CONTINUED)

2.3 Stimulus and Timing

The clock starts low and toggles every 10 ns, giving a 20 ns period. The top-level initialization asserts reset at time zero, so the first rising edge at 10 ns clears the counter before the driver applies its first queued transaction at 20 ns.

Transactions	Purpose
1	Reset to a known value.
2-3	Decrement zero to FFFF, then increment FFFF to zero.
4-5	Increment twice to 0001 and 0002.
6	Decrement to 0001.
7	Assert reset with up selected; check that reset wins.
8-31	Randomize the direction for 24 cycles with reset forced low.

The driver applies inputs at each falling edge. The monitor captures those inputs at the next rising edge, waits 1 ns for the DUT nonblocking update, and then reads count. With this 20 ns clock, the delay leaves ample time before the next falling edge. The scoreboard uses blocking assignments to calculate the expected state before comparing it with the sampled output using case equality (===).

Although rst is declared rand, the generator overwrites it with zero after randomization in the final 24 transactions. Thus the random phase randomizes counting direction only. Randomization does not guarantee that both directions appear in that phase; the directed sequence already exercises both.

The done event means that stimulus queueing has completed, not that the driver has consumed the queue. The top-level block then waits for 33 rising edges and an additional 2 ns. For this sequence, transaction 31 is applied at 620 ns and sampled at 630 ns; the final rising edge at 650 ns performs another count using the last direction. Simulation ends at 652 ns. The monitor also observes the initial reset edge, so sampled cycles and generated transactions are not one-to-one.

CHAPTER 3
RESULT ANALYSIS

3.1 Output Waveform

Figure 3.1 reproduces the supplied EPWave signal display. The horizontal axis is in picoseconds: 50,000 ps corresponds to 50 ns. The count bus is displayed in hexadecimal. The repeated signal groups show the same named clock, count, reset and direction signals at different hierarchy selections.

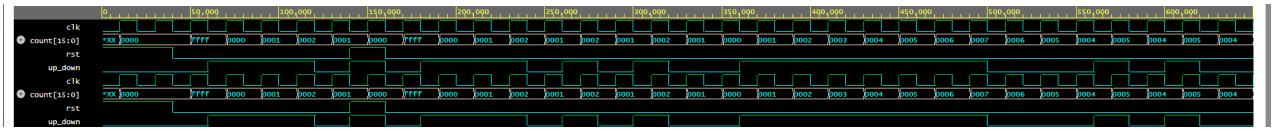


Figure 3.1: Supplied counter waveform (signal area).

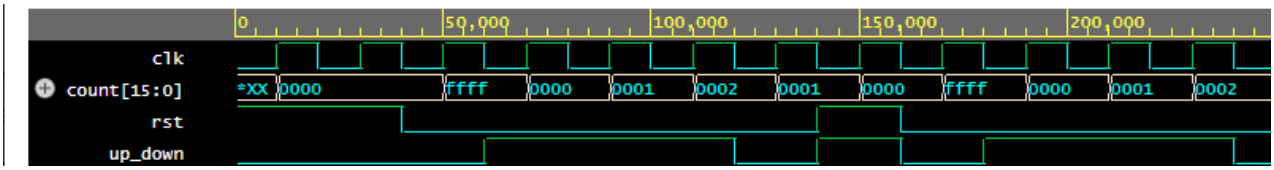


Figure 3.2: Enlarged initial sequence showing reset and wraparound.

3.2 Directed Sequence Analysis

Rising edge	rst	Direction	Expected count
10 ns	1	Down	0000 - initial reset
30 ns	1	Down	0000 - queued reset
50 ns	0	Down	FFFF - underflow
70 ns	0	Up	0000 - overflow
90 ns	0	Up	0001
110 ns	0	Up	0002
130 ns	0	Down	0001
150 ns	1	Up	0000 - reset priority

The displayed initial unknown count is consistent with an uninitialized register before the first rising edge. It becomes zero at the first active reset edge. The directed values shown in the waveform agree with the sequence predicted by the supplied code.

CHAPTER 3
RESULT ANALYSIS (CONTINUED)

3.3 Verification Scope

After the directed tests, the waveform shows alternating increments and decrements and a longer upward sequence reaching 0007 before counting downward. These transitions are consistent with the displayed direction signal. Changes in reset and direction occur on falling edges, while count updates occur on rising edges, as intended by the testbench.

The scoreboard checks every monitored sample after reset initialization. It predicts zero during reset, adds one for upward counting and subtracts one for downward counting. Because expected_count is also 16 bits, its arithmetic has the same modulo-65,536 range as the specification.

The supplied image establishes the visible sequence but does not provide a simulator console transcript. Therefore, the report does not assert that a particular number of PASS messages was produced or that the entire simulation was error-free. The listings document the checking mechanism; they have not been independently simulated as part of preparing this report.

This is a focused functional test, not exhaustive verification. It does not traverse all 65,536 states, collect functional coverage, or deliberately drive X/Z inputs. The clock and reset pulse placement are fixed. The fixed end-of-test wait is sufficient for the present 31-item sequence and must be revised if the stimulus length changes.

Interpretation of reset

Reset assertion alone does not immediately modify count. In the directed sequence, reset is applied at the falling edge at 140 ns and the counter clears at the following rising edge at 150 ns. This timing distinguishes the implemented synchronous reset from an asynchronous reset.

CHAPTER 4
CONCLUSION

4.1 Work Conclusion

A 16-bit synchronous up/down counter and a class-based SystemVerilog verification environment have been documented. The design counts upward or downward on each rising clock edge, gives priority to an active-high synchronous reset and wraps naturally at the 16-bit boundaries.

The verification structure follows the JK flip-flop architecture, retaining a transaction class, generator, driver, monitor, scoreboard, virtual interfaces and typed mailboxes. The expected-value model is adapted from a one-bit JK state update to 16-bit increment/decrement arithmetic. Directed tests cover the principal boundary and reset behaviors, followed by randomized direction stimulus.

The supplied waveform is consistent with the directed reset, underflow, overflow and direction-reversal sequence. The scoreboard provides automated comparison for each observed clock cycle. Future extensions could add functional coverage, assertions for reset and count transitions, longer randomized sequences and completion tracking based on processed transactions instead of a fixed cycle wait.

Code organization for EDA Playground

Place Annexure A.1 in **design.sv**. Place A.2 through A.8, in that order, in **testbench.sv**. Define module counter_16 only once. The counter_16 dut instance inside tb must remain: it connects the design to the interface and is not a second module definition.

Alternatively, the listings can be combined into one SystemVerilog source file if no other source defines counter_16. The commented code preserves the supplied behavior; comments explain declarations, object creation, mailbox transfers, timing and expected-value checks.

ANNEXURE A
SYSTEMVERILOG CODE

A.1 RTL - design.sv

```
`timescale 1ns/1ps                 // Time unit: 1 ns; precision: 1 ps.

module counter_16 (
    input  logic        clk,       // Common clock for all 16 bits.
    input  logic        rst,       // Active-high synchronous reset.
    input  logic        up_down,   // 1: increment; 0: decrement.
    output logic [15:0] count      // Unsigned 16-bit counter output.
);
    always_ff @(posedge clk) begin // Update only at each rising edge.
        if (rst)                   // Reset takes priority over counting.
            count <= 16'h0000;     // Clear all bits to zero.
        else if (up_down)          // Select upward counting.
            count <= count + 16'd1; // FFFF + 1 wraps to 0000.
        else                       // Select downward counting.
            count <= count - 16'd1; // 0000 - 1 wraps to FFFF.
    end                            // End sequential logic.
endmodule                          // End counter design.
```

ANNEXURE A
SYSTEMVERILOG CODE

A.2 Interface - start of testbench.sv

```
`timescale 1ns/1ps                  // Apply timing units to testbench.

interface counter_if;              // Group DUT and testbench signals.
    logic        clk;              // Clock generated by module tb.
    logic        rst;              // Reset driven by the driver.
    logic        up_down;          // Direction driven by the driver.
    logic [15:0] count;             // DUT output read by the monitor.
endinterface
```

A.3 Transaction class

```
class transaction;                 // Container for one test/sample.
    rand logic        rst;         // Randomizable reset field.
    rand logic        up_down;     // Randomizable direction field.
         logic [15:0] count;       // Observed output; not randomized.

    function void display();       // Print values without a time delay.
        // Display reset, direction and count in decimal and hexadecimal.
        $display("rst=%0b up_down=%0b count=%0d (0x%04h)",
                 rst, up_down, count, count);
    endfunction

    function transaction copy();  // Return an independent transaction.
        copy = new();              // Allocate a new object.
        copy.rst     = this.rst;   // Copy reset from the current object.
        copy.up_down = this.up_down; // Copy the counting direction.
        copy.count   = this.count; // Copy the observed counter value.
    endfunction                    // Function name holds return object.
endclass
```

ANNEXURE A
SYSTEMVERILOG CODE

A.4 Generator class

```
class generator;                  // Build the stimulus sequence.
    transaction T1;               // Handle to reusable stimulus object.
    mailbox #(transaction) G2D;   // Typed generator-to-driver mailbox.
    event done;                   // Indicates all inputs are queued.

    function new(mailbox #(transaction) G2D);
        this.G2D = G2D;           // Store the shared mailbox handle.
        T1 = new();               // Allocate the stimulus object.
    endfunction

    task run();                   // Generate directed and random tests.
        // Transaction 1: establish a known state using reset.
        T1.rst = 1;               // Assert reset.
        T1.up_down = 0;           // Direction is ignored during reset.
        G2D.put(T1.copy());       // Queue a separate snapshot.

        // Transaction 2: check underflow from 0000 to FFFF.
        T1.rst = 0;               // Release reset.
        T1.up_down = 0;           // Request one decrement.
        G2D.put(T1.copy());       // Send the underflow test.

        // Transaction 3: check overflow from FFFF to 0000.
        T1.up_down = 1;           // Reverse direction to count up.
        G2D.put(T1.copy());       // Send the overflow test.

        // Transactions 4 and 5: count from 0000 to 0001 to 0002.
        repeat (2) begin          // Queue two increment operations.
            T1.up_down = 1;       // Keep upward counting selected.
            G2D.put(T1.copy());   // Preserve each queued transaction.
        end

        // Transaction 6: reverse direction, 0002 to 0001.
        T1.up_down = 0;           // Request downward counting.
        G2D.put(T1.copy());       // Queue the decrement operation.

        // Transaction 7: reset a nonzero count back to 0000.
        T1.rst = 1;               // Reset must override counting.
        T1.up_down = 1;           // Up is selected to test priority.
        G2D.put(T1.copy());       // Queue the second reset.

        // Transactions 8-31: exercise randomized counting directions.
        repeat (24) begin         // Generate 24 random transactions.
            if (!T1.randomize()) // Randomize the rand fields.
                $fatal(1, "Transaction randomization failed");
            T1.rst = 0;           // Override reset: only direction varies.
            G2D.put(T1.copy());   // Send a stable copy to the driver.
        end
        -> done;                  // Queueing finished, not driving.
    endtask
endclass
```

ANNEXURE A
SYSTEMVERILOG CODE

A.5 Driver class

```
class driver;                      // Convert transactions to signals.
    virtual counter_if intf;       // Handle to the actual interface.
    mailbox #(transaction) G2D;    // Receives generator transactions.
    transaction T1;                // Handle to the received transaction.

    function new(mailbox #(transaction) G2D);
        this.G2D = G2D;            // Connect to the shared mailbox.
    endfunction

    task run();                    // Driver operates throughout the test.
        forever begin              // Repeat until simulation ends.
            G2D.get(T1);           // Block until a transaction arrives.
            @(negedge intf.clk);   // Drive before the next rising edge.
            intf.rst <= T1.rst;    // Apply reset using a nonblocking write.
            intf.up_down <= T1.up_down; // Apply the direction input.

            // Report only input values: the driver does not sample count.
            $display("DRIVER: rst=%0b up_down=%0b",
                     T1.rst, T1.up_down);
        end
    endtask
endclass
```

ANNEXURE A
SYSTEMVERILOG CODE

A.6 Monitor class

```
class monitor;                    // Observe inputs and DUT response.
    virtual counter_if intf;      // Handle to the shared interface.
    mailbox #(transaction) M2S;   // Monitor-to-scoreboard mailbox.
    transaction T1;               // Handle for each observed sample.

    function new(mailbox #(transaction) M2S);
        this.M2S = M2S;           // Store the observation mailbox.
    endfunction

    task run();
        forever begin             // Observe every active clock edge.
            @(posedge intf.clk);  // Same edge that updates the DUT.
            T1 = new();           // Allocate a fresh sample object.
            T1.rst = intf.rst;    // Capture reset used at this edge.
            T1.up_down = intf.up_down; // Capture the active direction.

            #1;                   // Wait 1 ns for the DUT NBA update.
            T1.count = intf.count; // Sample the updated counter output.
            $display("MONITOR: Observed transaction");
            T1.display();         // Print the captured transaction.
            M2S.put(T1.copy());   // Send an independent sample to check.
        end
    endtask
endclass
```

ANNEXURE A
SYSTEMVERILOG CODE

A.7 Scoreboard class

```
class scoreboard;                 // Predict and check counter behavior.
    mailbox #(transaction) M2S;   // Receives monitor observations.
    transaction T1;               // Handle to the latest observation.
    logic [15:0] expected_count;  // Reference state; wraps at 16 bits.
    bit initialized = 0;          // Checking requires an observed reset.

    function new(mailbox #(transaction) M2S);
        this.M2S = M2S;           // Connect to the monitor mailbox.
        expected_count = 16'h0000; // Initial reference value.
    endfunction

    task run();
        forever begin             // Check every received observation.
            M2S.get(T1);          // Wait until a sample is available.
            if (T1.rst) begin     // Reset overrides direction.
                expected_count = 16'h0000; // Predict cleared output.
                initialized = 1; // A known reference state now exists.
            end
            else if (initialized) begin // Predict only after reset.
                if (T1.up_down)  // Upward counting selected.
                    expected_count = expected_count + 16'd1;
                else             // Downward counting selected.
                    expected_count = expected_count - 16'd1;
            end
            else begin            // No reset observed yet.
                $display("SCOREBOARD: Waiting for reset");
                continue;         // Skip comparison for this sample.
            end

            // Case equality produces 0 or 1, even with X/Z in count.
            if (T1.count === expected_count)
                $display("PASS: Expected=%0d, Actual=%0d",
                         expected_count, T1.count);
            else                  // Flag an incorrect DUT response.
                $error("FAIL: Expected=%0d, Actual=%0d",
                       expected_count, T1.count);
        end
    endtask
endclass
```

ANNEXURE A
SYSTEMVERILOG CODE

A.8 Top-level testbench

```
module tb;                        // Instantiate and connect everything.
    counter_if intf();            // Create the physical interface.
    generator  gen;               // Generator object handle.
    driver     drv;               // Driver object handle.
    monitor    mon;               // Monitor object handle.
    scoreboard soc;               // Scoreboard object handle.
    mailbox #(transaction) G2D;   // Generator-driver communication.
    mailbox #(transaction) M2S;   // Monitor-scoreboard communication.

    counter_16 dut (              // Instantiate the separately defined DUT.
        .clk     (intf.clk),      // Connect clock input.
        .rst     (intf.rst),      // Connect synchronous reset.
        .up_down (intf.up_down),  // Connect direction control.
        .count   (intf.count)     // Connect the counter output.
    );

    initial intf.clk = 1'b0;      // Start clock low at time zero.
    always #10 intf.clk = ~intf.clk; // Toggle every 10 ns: period 20 ns.

    initial begin                 // Establish defined initial inputs.
        intf.rst = 1'b1;          // First rising edge resets the DUT.
        intf.up_down = 1'b0;      // Initial direction: down.
    end

    initial begin                 // Construct the verification setup.
        G2D = new();              // Allocate an unbounded typed mailbox.
        M2S = new();              // Allocate the observation mailbox.
        gen = new(G2D);           // Give generator the G2D handle.
        drv = new(G2D);           // Give driver the same G2D handle.
        mon = new(M2S);           // Give monitor the M2S handle.
        soc = new(M2S);           // Give scoreboard the same M2S handle.
        drv.intf = intf;          // Bind driver's virtual interface.
        mon.intf = intf;          // Bind monitor's virtual interface.

        fork                      // Start concurrent verification tasks.
            gen.run();            // Generate and queue stimulus.
            drv.run();            // Apply one transaction per cycle.
            mon.run();            // Sample each DUT response.
            soc.run();            // Predict and compare outputs.
        join_none                 // Parent continues without waiting.

        wait (gen.done.triggered); // Wait for all 31 items to be queued.
        repeat (33) @(posedge intf.clk); // Allow queue to drain and check.
        #2;                       // Allow final monitor/check to finish.
        $finish;                  // End all simulation processes.
    end

    initial begin                 // Enable waveform recording.
        $dumpfile("counter_dump.vcd"); // Name the waveform file.
        $dumpvars(0, tb);          // Dump tb and its hierarchy.
    end
endmodule
```

EDA Playground Code Link

https://edaplayground.com/x/kLmW